

TP 1 — DevOps

Mise en place d'une chaîne CI/CD : GitHub → GitHub Actions → Render

Objectif	Créer une application web, la versionner sur GitHub et la déployer automatiquement sur Render, puis démontrer le redéploiement continu (CI/CD).
Étudiant	Anthony Ferron (AnthonyFerron)
Cours	3ème année — Méthodes Agiles / DevOps
Date	10 juin 2026
Dépôt GitHub	https://github.com/AnthonyFerron/tp1-devops
Application en ligne	https://tp1-devops-6vgr.onrender.com

Chaîne DevOps réalisée

Code → GitHub → GitHub Actions (CI) → Build → Render → Production

1. Synthèse des étapes

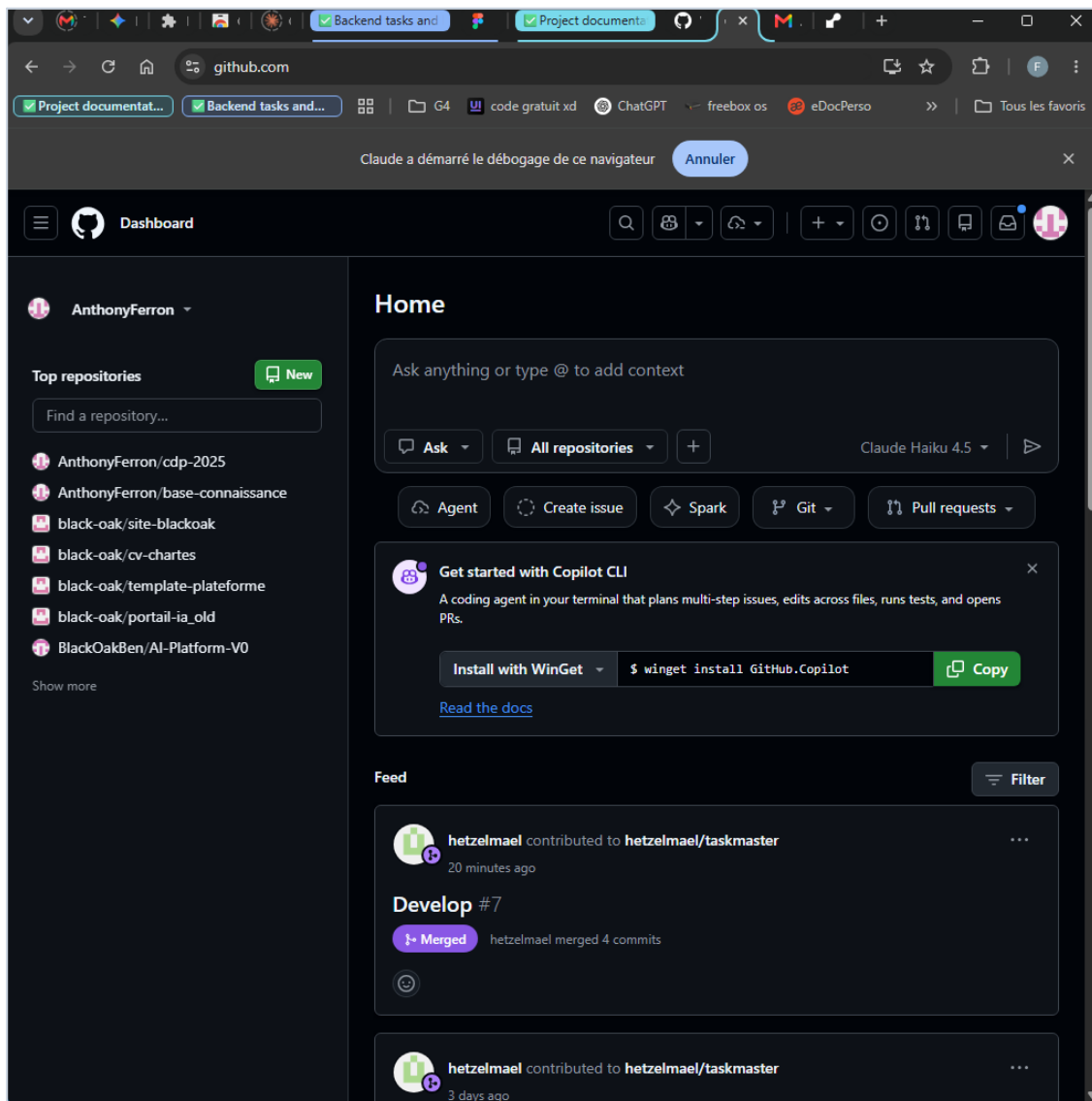
Le tableau ci-dessous résume les huit étapes du TP et leur statut. Chaque étape est détaillée dans les sections suivantes, preuve (capture d'écran) à l'appui.

#	Étape	Statut
1	Création des comptes GitHub et Render	✓ Réalisé
2	Création de l'application Node.js / Express (app.js)	✓ Réalisé
3	Création du dépôt GitHub et premier push	✓ Réalisé
4	Connexion Render ↔ GitHub	✓ Réalisé
5	Création du Web Service (build / start)	✓ Réalisé
6	Premier déploiement automatique (URL en ligne)	✓ Réalisé
7	Démonstration CI/CD (modification → push → redéploiement auto)	✓ Réalisé
8	Ajout de GitHub Actions (.github/workflows/main.yml)	✓ Réalisé

2. Étape 1 — Création des comptes GitHub et Render

La première étape consiste à disposer d'un compte **GitHub** (hébergement du code et exécution de l'intégration continue) et d'un compte **Render** (plateforme de déploiement / hébergement de l'application). Le compte Render a été créé via « Sign in with GitHub », ce qui établit immédiatement le lien entre les deux services (cf. étape 4).

- Compte GitHub : **AnthonyFerron** — connecté et opérationnel.
- Compte Render : créé via GitHub OAuth, e-mail vérifié.



■ Étape 1 — Tableau de bord GitHub, connecté en tant que « AnthonyFerron ».

3. Étape 2 — Application Node.js / Express

Une application web minimale a été développée avec **Express**. Elle expose une route racine « / » renvoyant le message « Bonjour DevOps ! ».

app.js

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Bonjour DevOps !");
});

// Render fournit le port via la variable d'environnement PORT.
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Serveur démarré sur le port ${PORT}`);
});
```

Remarque technique importante : l'énoncé utilise `app.listen(3000)` en dur. Or Render impose à l'application d'écouter sur le port fourni par la variable d'environnement `PORT`. Le code a donc été adapté avec `process.env.PORT || 3000` ; sans cela le déploiement échouerait (le port reste 3000 en repli pour le développement local).

package.json

```
{
  "name": "tp1-devops",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": { "start": "node app.js" },
  "dependencies": { "express": "^4.21.2" }
}
```

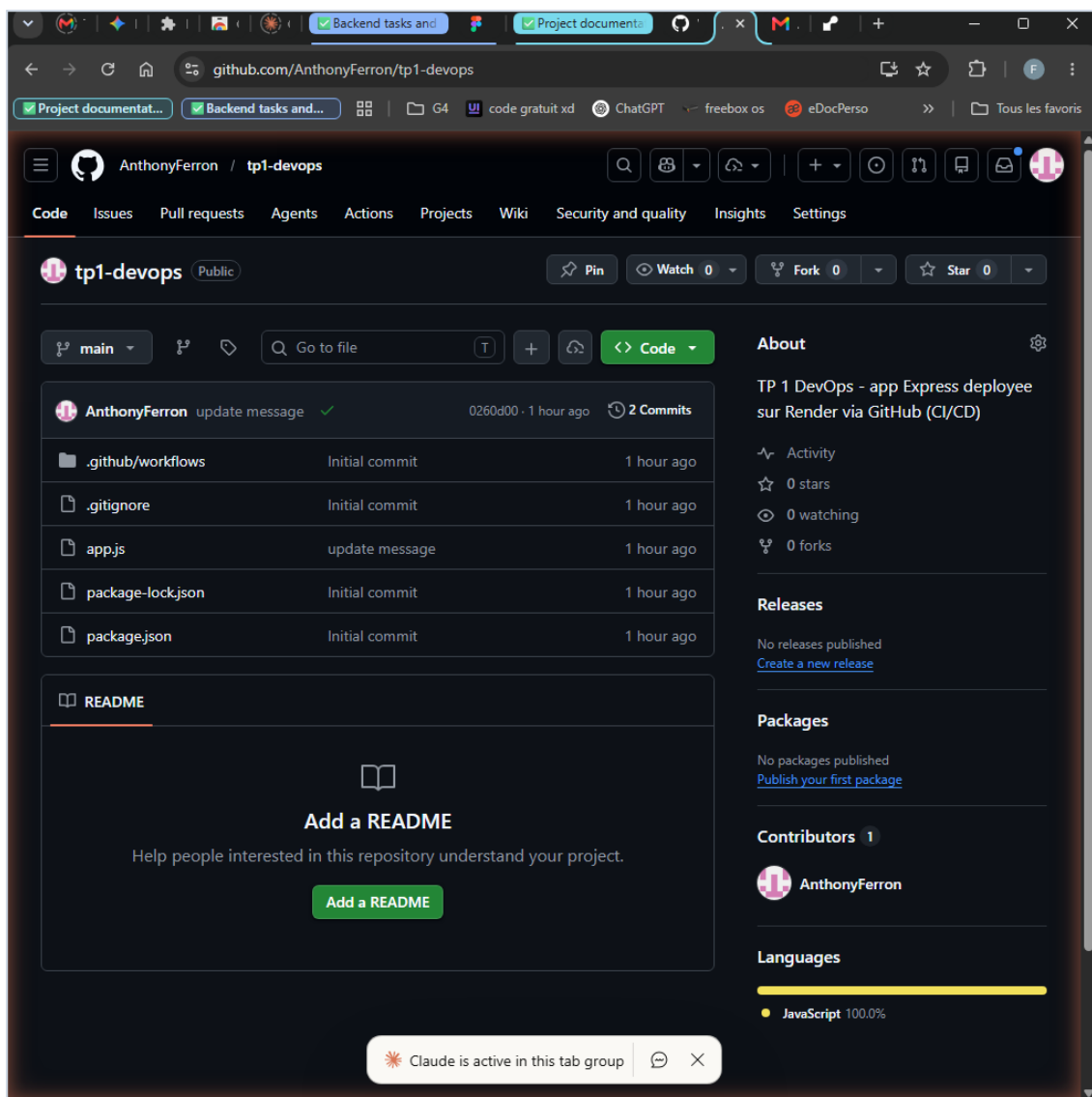
4. Étape 3 — Dépôt GitHub et premier push

Le projet a été initialisé en dépôt Git puis poussé vers un nouveau dépôt public **tp1-devops** sur GitHub.

Commandes exécutées

```
git init -b main
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/AnthonyFerron/tp1-devops.git
git push -u origin main
```

- Le fichier `.gitignore` exclut `node_modules/` et `.env` du suivi.
- Fichiers versionnés : `app.js`, `package.json`, `package-lock.json`, `.gitignore`, `.github/workflows/main.yml`.



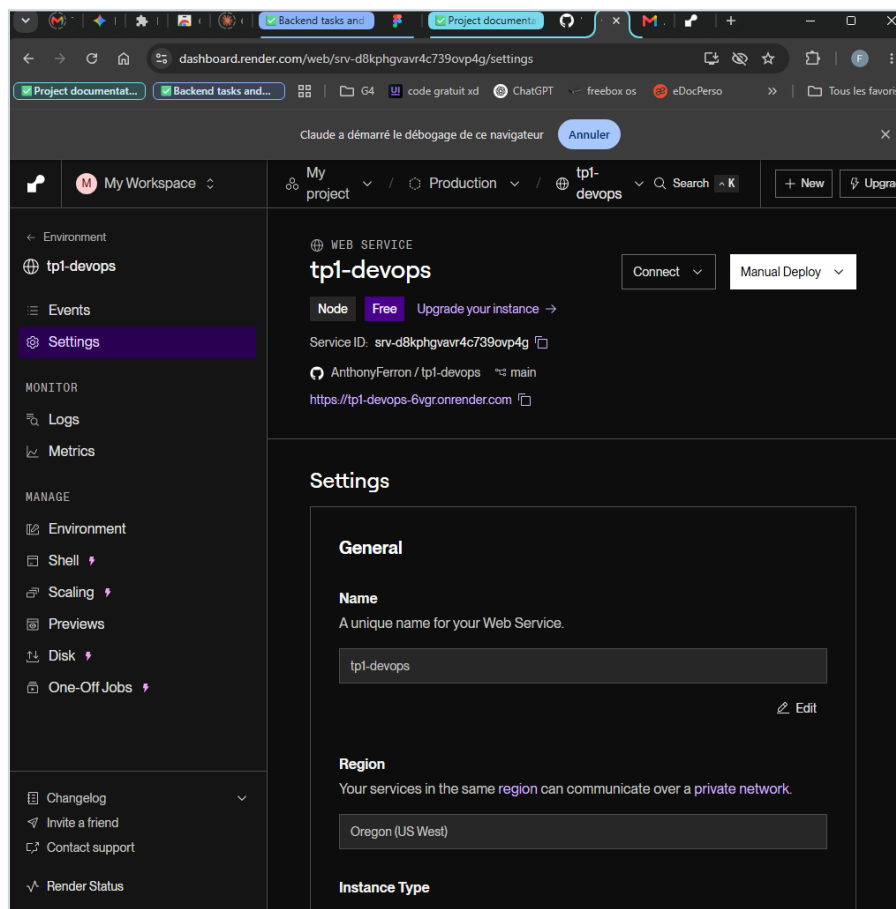
- Étape 3 — Dépôt « *tp1-devops* » sur GitHub avec l'ensemble des fichiers (commit « *update message* », 2 commits).

5. Étapes 4 & 5 — Render ↔ GitHub et Web Service

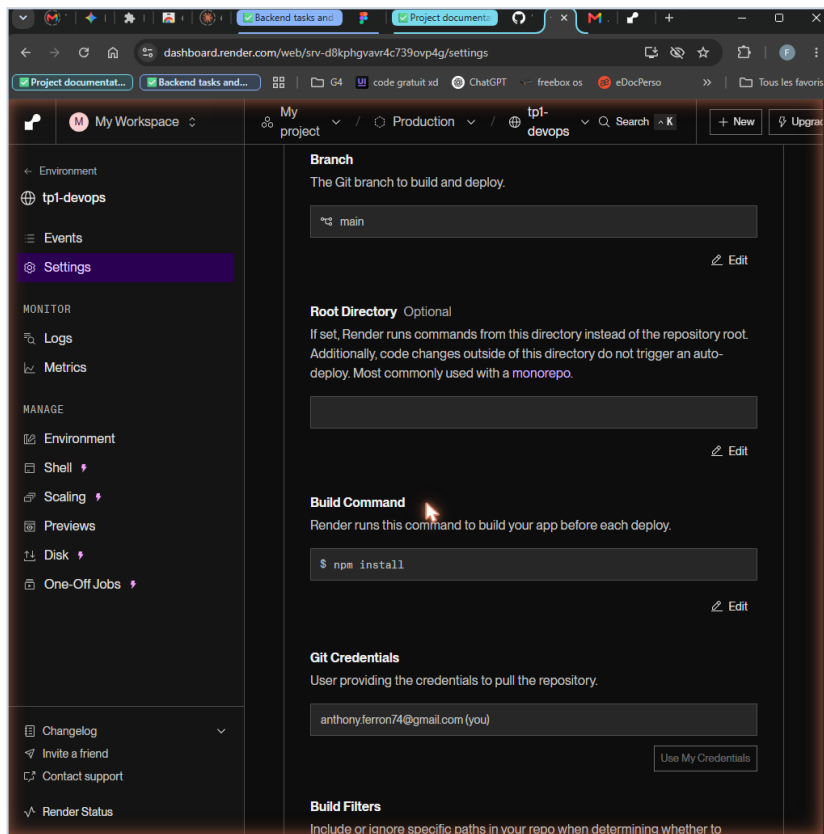
Sur Render, un **Web Service** a été créé à partir du dépôt GitHub **tp1-devops**. Render a détecté automatiquement un projet **Node** et pré-rempli la configuration. L'offre **Free** (0,1 CPU / 512 Mo) a été choisie.

Configuration du service

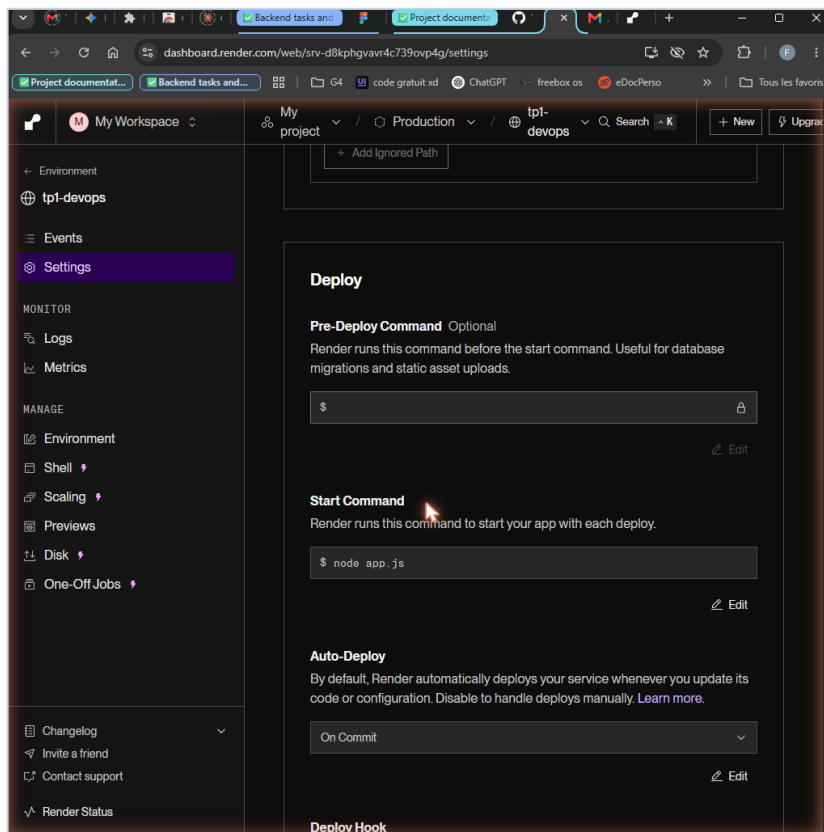
Paramètre	Valeur
Dépôt source	AnthonyFerron/tp1-devops (branche main)
Langage	Node
Build Command	npm install
Start Command	node app.js
Type d'instance	Free — 0,1 CPU / 512 Mo
Région	Oregon (US West)
Auto-Deploy	On Commit (redéploiement à chaque push)



■ Étapes 4/5 — Web Service Render « tp1-devops » : type Node, offre Free, dépôt GitHub lié et URL de production.



■ Étape 5 — Build Command : « npm install » (conforme à l'énoncé).



■ Étape 5/7 — Start Command : « node app.js » et Auto-Deploy réglé sur « On Commit ».

6. Étape 6 — Premier déploiement

Render a lancé automatiquement le premier build puis le déploiement. L'application est devenue accessible publiquement à l'adresse :

<https://tp1-devops-6vgr.onrender.com>

- Build : `npm install` — Démarrage : `node app.js`.
- Statut « **Deploy live** » obtenu pour le commit « Initial commit » (73ffad6).
- Vérification HTTP : la page répond **200** avec « Bonjour DevOps ! ».

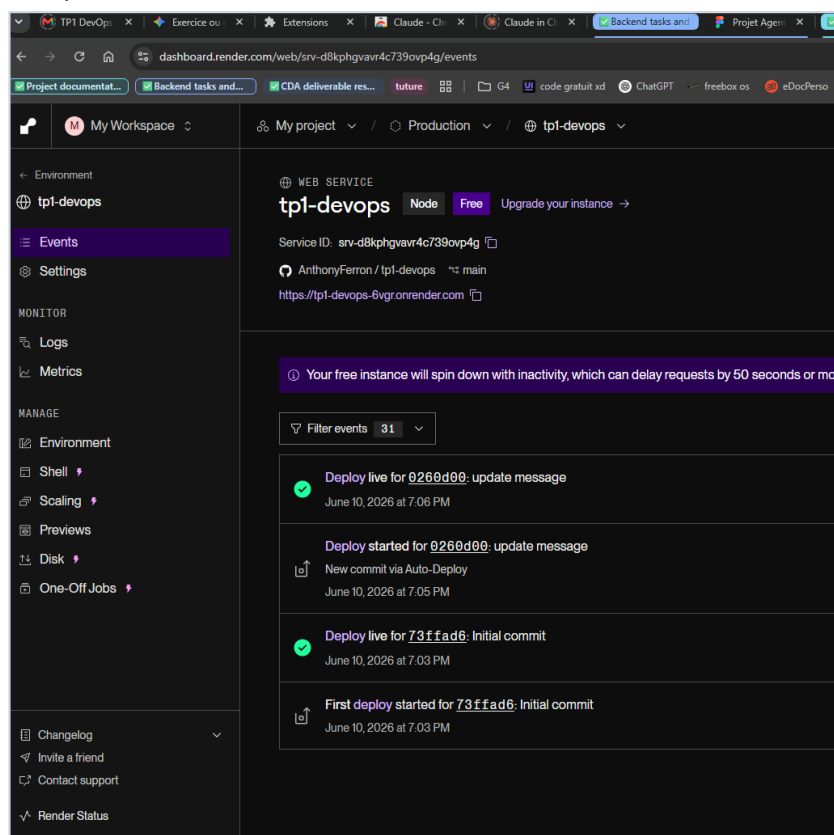
7. Étape 7 — Démonstration CI/CD (cœur du TP)

Pour démontrer l'esprit DevOps, le message a été modifié dans le code source, « **Bonjour DevOps !** » → « **Bonjour DevOps 2026 !** », puis simplement poussé sur GitHub :

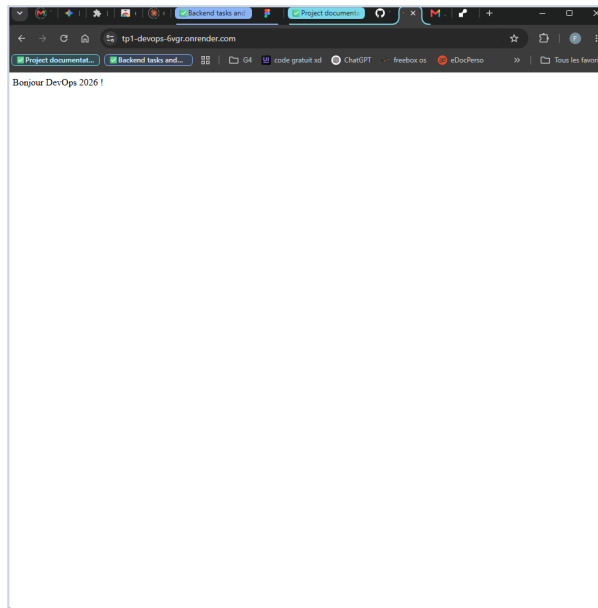
```
git add .
git commit -m "update message"
git push
```

Sans aucune action manuelle sur Render, le push a déclenché toute la chaîne : nouveau commit détecté → nouveau build → nouveau déploiement → nouveau site en ligne. C'est exactement l'avantage du CI/CD.

- Render affiche « **Deploy started for 0260d00 — New commit via Auto-Deploy** » puis « **Deploy live** ».
- La page en ligne affiche désormais « Bonjour DevOps 2026 ! » (contenu mis à jour automatiquement).



■ Étape 7 — Historique des déploiements Render : « Initial commit » (live), puis « New commit via Auto-Deploy » → « update message » (live). Aucune action manuelle.



■ *Étape 7 — Application en ligne après le push : « Bonjour DevOps 2026 ! » sur l'URL onrender.com.*

8. Étape 8 — GitHub Actions (CI)

Un workflow d'intégration continue a été ajouté. À chaque push, GitHub Actions récupère le code, installe Node 20 et exécute `npm install` — ce qui valide que le projet se construit correctement.

[.github/workflows/main.yml](#)

```
name: CI

on:
  push:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm install
```

Résultat observé dans l'onglet **Actions** du dépôt : **2 exécutions du workflow CI**, toutes deux **réussies (vertes)** :

Run	Commit	Déclencheur	Statut	Durée
CI #1	73ffad6 — Initial commit	push	✓ Réussi	~13 s
CI #2	0260d00 — update message	push	✓ Réussi	~14 s

9. Conclusion

L'ensemble des objectifs du TP a été atteint. Une chaîne DevOps complète et fonctionnelle a été mise en place :

Code → GitHub → GitHub Actions (CI) → Build → Render → Production

La démonstration clé (étape 7) prouve l'automatisation complète : une simple modification du code, validée par un `git push`, déclenche l'intégration continue (GitHub Actions) et le déploiement continu (Render) **sans aucune intervention manuelle**. C'est précisément la valeur ajoutée du CI/CD.